

SOFTWARE REQUIREMENTS SPECIFICATION

SkyCast

A Smart Weather Prediction System

Document Type: Software Requirements Specification (SRS)

Prepared as per IEEE 830 / IEEE 29148 SRS Guidelines

Version: 1.0

Date: 18 August 2026

Field	Details
Project Title	SkyCast – Smart Weather Prediction System
Prepared For	GLA UNIVERSITY
Technology Stack	HTML CSS3 JavaScript(Es6)
Prepared By	ANSHIKA SHARMA

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 References.....	3
1.5 Document Overview.....	3
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions	5
2.3 User Classes and Characteristics	5
2.4 Operating Environment.....	5
2.5 Design and Implementation Constraints.....	5
2.6 Assumptions and Dependencies	6
3. System Features (Functional Requirements)	7
3.1 User Module.....	7
3.2 Admin Module.....	9
4. External Interface Requirements	11
4.1 User Interfaces	11
4.2 Hardware Interfaces.....	11
4.3 Software Interfaces	11
4.4 Communication Interfaces	11
5. Non-Functional Requirements.....	12
5.1 Performance Requirements	12
5.2 Security Requirements	12
5.3 Usability Requirements	12
5.4 Reliability and Availability	12
5.5 Maintainability	12
5.6 Scalability.....	12
6. System Architecture	13
7. Data Requirements.....	14
8. Use Case Summary	15
9. Software Testing Overview	16
10. Assumptions, Constraints, and Future Enhancements	17
10.1 Assumptions	17
10.2 Constraints	17
10.3 Future Enhancements	17
11. Appendix.....	18
11.1 Revision History.....	18

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for “SkyCast – Smart Weather Prediction System”, a frontend-based web application that allows users to check current weather conditions, view short-term forecasts, and manage location-based weather preferences. The document is intended to serve as the primary reference for the development team, project guide, and evaluating faculty throughout the design, implementation, testing, and presentation phases of the project.

1.2 Scope

SkyCast is a web-based weather information system consisting of two primary modules: a User module and an Admin module. The User module enables registered and guest users to search weather information by city or location, view current conditions and a multi-day forecast, save favourite locations, and receive basic weather alerts. The Admin module allows an administrator to manage registered users, configure the weather data source, monitor system usage, and broadcast alert notifications. The system is designed as a realistic, achievable academic project that demonstrates core software engineering concepts such as requirement analysis, modular design, API integration, and database-driven web development, without extending into enterprise-scale complexity such as IoT sensor networks or proprietary meteorological modelling.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
SRS	Software Requirements Specification
API	Application Programming Interface
UI/UX	User Interface / User Experience
GUI	Graphical User Interface
DB	Database
Admin	Administrator – the privileged system user
JSON	JavaScript Object Notation, data format used by the weather API
HTTPS	Hypertext Transfer Protocol Secure
CRUD	Create, Read, Update, Delete

1.4 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 29148-2018 – Systems and Software Engineering – Requirements Engineering.
- OpenWeatherMap API Documentation (or equivalent public weather data API), used as the third-party weather data source.
- College Software Engineering course guidelines for final year / semester project documentation.

1.5 Document Overview

The remainder of this document is organised as follows: Section 2 provides an overall description of the product, its users and operating environment. Section 3 details the system features and functional requirements grouped by module. Section 4 specifies external interface requirements. Section 5 lists non-functional requirements. Section 6 presents the proposed system architecture. Section 7 describes data requirements. Section 8 summarises key use cases. Section 9 outlines the testing approach, and Section 10 records assumptions, constraints and planned future enhancements.

2. Overall Description

2.1 Product Perspective

SkyCast is a new, self-contained academic web application. It is not a modification of an existing system. The frontend will be built using standard web technologies (HTML, CSS, JavaScript, and a modern framework such as React), while the backend will expose a lightweight REST API that communicates with a relational database and consumes a third-party public weather API for live forecast data. The system follows a typical three-tier client-server architecture, described further in Section 6.

2.2 Product Functions

At a high level, SkyCast provides the following functions:

- User registration, login, and profile management.
- Search and display of current weather conditions for a city or location.
- Display of a short-term (up to 5-day) weather forecast.
- Saving and managing a list of favourite / frequently checked locations.
- Basic weather alert notifications (e.g., heavy rain, extreme temperature) for saved locations.
- Admin management of registered users and system configuration.
- Admin monitoring of system usage logs and basic analytics.
- Admin broadcast of manual alert notifications to all users.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Guest User	Can search and view current weather and forecast without creating an account.	Minimal – general public
Registered User	Can log in, save favourite locations, and receive alerts.	Minimal – general public
Administrator	Manages users, data source configuration, and system-wide alerts.	Moderate – comfortable with web dashboards

2.4 Operating Environment

- Client side: any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile, screen width ≥ 360 px.
- Server side: Node.js runtime with a REST API layer, deployable on a standard cloud or campus server.
- Database: relational database such as MySQL or PostgreSQL (or a lightweight alternative such as SQLite for demonstration).
- External dependency: a public weather data API accessed over HTTPS.

2.5 Design and Implementation Constraints

- The project must be completed within a single academic semester by a small student team.
- Live weather data is limited by the rate limits and accuracy of the free tier of the chosen third-party weather API.
- The system will not perform its own meteorological forecasting/modelling; it consumes and presents third-party forecast data.

- The application is frontend-focused; backend/API and database form a supporting layer sized for demonstration, not production-scale traffic.

2.6 Assumptions and Dependencies

- Users have a working internet connection while using the application.
- The third-party weather API remains available and free-to-use for academic purposes during the project timeline.
- Faculty evaluation will be based on a working demonstration rather than production deployment at scale.

3. System Features (Functional Requirements)

This section describes the functional requirements of SkyCast, grouped by the User module and the Admin module. Each feature includes a short description, priority, and associated requirement identifiers for traceability.

3.1 User Module

3.1.1 Registration and Login

Allows a new visitor to create an account and existing users to log in securely to access personalised features such as saved locations and alerts.

Priority: High

Req. ID	Requirement Description	Notes
FR-01	The system shall allow a new user to register using name, email, and password.	Email must be unique
FR-02	The system shall allow a registered user to log in using email and password.	Session/token-based auth
FR-03	The system shall allow a user to log out of the application.	Clears active session

3.1.2 Search Weather by Location

Allows any user (guest or registered) to search for a city or location and view its weather information.

Priority: High

Req. ID	Requirement Description	Notes
FR-04	The system shall provide a search bar to enter a city or location name.	Auto-suggest optional
FR-05	The system shall fetch live weather data for the searched location from the weather API.	Depends on API availability
FR-06	The system shall display an appropriate message if the location is not found.	Error handling

3.1.3 Current Weather Display

Displays the current temperature, weather condition, humidity, wind speed, and related details for the selected location.

Priority: High

Req. ID	Requirement Description	Notes
FR-07	The system shall display current temperature in Celsius (with an option for Fahrenheit).	Unit toggle
FR-08	The system shall display humidity, wind speed, and general condition (e.g., cloudy, rainy).	Sourced from API response
FR-09	The system shall display a relevant weather icon based on current conditions.	Improves readability

3.1.4 5-Day Weather Forecast

Displays a short-term forecast to help users plan ahead.

Priority: Medium

Req. ID	Requirement Description	Notes
FR-10	The system shall display a day-wise forecast for up to 5 days for the selected location.	Summarised daily values
FR-11	The system shall show minimum and maximum expected temperature per day.	From forecast API

3.1.5 Favourite Locations

Allows a registered user to save frequently checked locations for quick access.

Priority: Medium

Req. ID	Requirement Description	Notes
FR-12	The system shall allow a registered user to add a location to a favourites list.	Login required
FR-13	The system shall allow a registered user to remove a location from favourites.	Login required
FR-14	The system shall display saved favourite locations on the user dashboard.	Quick-access panel

3.1.6 Weather Alerts and Notifications

Notifies users of significant weather conditions for their saved locations.

Priority: Low

Req. ID	Requirement Description	Notes
FR-15	The system shall display an in-app alert when a saved location has severe weather (e.g., heavy rain, storm).	Basic threshold-based rule
FR-16	The system shall allow a user to dismiss an alert.	Per-session dismissal

3.1.7 Historical Weather View

Provides users a simple view of recently searched weather data for reference.

Priority: Low

Req. ID	Requirement Description	Notes
FR-17	The system shall store and display a user's last 10 searched locations.	Stored per account

3.2 Admin Module

3.2.1 Admin Login

Provides a secure, separate login for the administrator to access the admin dashboard.

Priority: High

Req. ID	Requirement Description	Notes
FR-18	The system shall authenticate the administrator using a dedicated admin login.	Role-based access control

3.2.2 Manage Users

Allows the administrator to view and manage registered user accounts.

Priority: Medium

Req. ID	Requirement Description	Notes
FR-19	The system shall allow the admin to view a list of all registered users.	Paginated list
FR-20	The system shall allow the admin to activate or deactivate a user account.	Soft-delete preferred

3.2.3 Manage Data Source Configuration

Allows the administrator to configure basic settings related to the weather API integration.

Priority: Low

Req. ID	Requirement Description	Notes
FR-21	The system shall allow the admin to update the configured weather API key/endpoint.	Stored securely

3.2.4 System Usage Analytics

Provides the administrator a basic overview of system usage for monitoring purposes.

Priority: Low

Req. ID	Requirement Description	Notes
FR-22	The system shall display the total number of registered users.	Dashboard summary
FR-23	The system shall display the most frequently searched locations.	Simple aggregate query

3.2.5 Broadcast Alerts

Allows the administrator to manually send an alert notification to all users, e.g., for a regional weather warning.

Priority: Low

Req. ID	Requirement Description	Notes
FR-24	The system shall allow the admin to compose and broadcast an alert message to all registered users.	Visible on next login/session

4. External Interface Requirements

4.1 User Interfaces

- A responsive web interface with a Home/Search screen, Weather Detail screen, Favourites screen, and Login/Register screens for users.
- A separate Admin Dashboard interface with navigation for Users, Configuration, Analytics, and Alerts.
- Consistent, minimal, and legible UI design using a clean colour palette and clear typography, optimised for both desktop and mobile viewports.

4.2 Hardware Interfaces

The system does not require any specialised hardware. It runs on standard client devices (desktop, laptop, tablet, smartphone) with internet connectivity and a modern web browser.

4.3 Software Interfaces

Interface	Description
Weather Data API	Third-party REST API (e.g., OpenWeatherMap) providing current and forecast weather data in JSON format over HTTPS.
Database	Relational database (MySQL/PostgreSQL/SQLite) storing user accounts, favourites, search history, and admin configuration.
Backend REST API	Custom REST API layer built with Node.js/Express, mediating between the frontend, database, and weather API.

4.4 Communication Interfaces

- All client-server communication shall use HTTPS.
- Data exchange between frontend and backend, and between backend and the weather API, shall use JSON.

5. Non-Functional Requirements

5.1 Performance Requirements

- The system shall display search results within 3 seconds under normal network conditions.
- The system shall support at least 50 concurrent users for demonstration purposes without noticeable degradation.

5.2 Security Requirements

- Passwords shall be stored using a secure hashing algorithm (e.g., bcrypt); plaintext passwords shall never be stored.
- Admin functionality shall be protected by role-based access control, separate from regular user accounts.
- All API keys and sensitive configuration shall be stored server-side and never exposed to the client.

5.3 Usability Requirements

- The interface shall be intuitive enough for a first-time user to search and view weather information without instructions.
- The system shall be responsive and usable on both desktop and mobile screen sizes.

5.4 Reliability and Availability

- The system shall handle API failures gracefully by displaying a clear error message instead of crashing.
- The system shall target 95% uptime during the evaluation and demonstration period.

5.5 Maintainability

- The codebase shall follow a modular structure (separate components for search, forecast, favourites, and admin) to simplify future maintenance.
- Configuration values (e.g., API endpoint) shall be externalised rather than hard-coded.

5.6 Scalability

- The architecture shall allow the addition of new features (e.g., push notifications, multi-language support) in future phases without major redesign.

6. System Architecture

SkyCast follows a standard three-tier architecture consisting of a Presentation Layer, an Application/API Layer, and a Data Layer, as summarised below.

Layer	Description
Presentation Layer	Web frontend (HTML/CSS/JavaScript, React) responsible for UI rendering, user interaction, and calling backend APIs.
Application / API Layer	Node.js/Express REST API handling authentication, business logic, favourites management, and communication with the external weather API.
Data Layer	Relational database storing users, favourite locations, search history, and admin configuration; the live weather data itself is fetched on demand from the external API rather than stored permanently.

This layered approach keeps the frontend, backend, and data concerns independent, which simplifies development by a small student team, supports parallel work on UI and API, and keeps the system easy to test and extend.

7. Data Requirements

The following core entities are proposed for the SkyCast database. This is intended as a starting point for detailed database/API design in a later project phase.

Entity	Key Attributes	Notes
User	user_id, name, email, password_hash, created_at	One user can have many favourites
FavouriteLocation	favourite_id, user_id, city_name, latitude, longitude	Linked to User by user_id
SearchHistory	history_id, user_id, city_name, searched_at	Stores last 10 searches per user
Admin	admin_id, username, password_hash	Separate table for role separation
AlertBroadcast	alert_id, message, created_by, created_at	Created by Admin, visible to all users

8. Use Case Summary

The table below summarises the primary use cases of the system. Detailed use case diagrams and flows can be developed in the design phase based on this list.

Use Case ID	Actor	Description
UC-01	Guest / User	Search and view current weather for a location.
UC-02	User	Register a new account.
UC-03	User	Log in to an existing account.
UC-04	User	View 5-day forecast for a location.
UC-05	User	Add or remove a favourite location.
UC-06	User	View and dismiss weather alerts.
UC-07	Admin	Log in to the admin dashboard.
UC-08	Admin	View and manage registered users.
UC-09	Admin	Configure weather API settings.
UC-10	Admin	Broadcast an alert to all users.

9. Software Testing Overview

The following testing approach is planned to validate SkyCast against the requirements in this document.

Testing Type	Purpose
Unit Testing	Verify individual functions/components (e.g., search input validation, temperature conversion) work as expected.
Integration Testing	Verify that the frontend, backend API, database, and external weather API work correctly together.
System Testing	Validate complete end-to-end flows such as search → display weather → save favourite.
User Acceptance Testing (UAT)	Confirm the system meets user expectations through demonstration to peers/faculty.
Regression Testing	Re-verify existing functionality after new features or bug fixes are added.

10. Assumptions, Constraints, and Future Enhancements

10.1 Assumptions

- Weather data accuracy depends entirely on the chosen third-party API and is outside the control of this project.
- The project will be demonstrated in a controlled environment with stable internet connectivity.

10.2 Constraints

- Limited development time (single semester) restricts scope to the features listed in Section 3.
- Free-tier API usage limits may restrict the number of live requests during heavy testing/demo.

10.3 Future Enhancements

- Push notifications for severe weather alerts via email or mobile notification.
- Support for multiple languages and units (metric/imperial) based on user locale.
- Integration of interactive weather maps (radar, satellite view).
- Machine-learning-based short-term local forecasting as an advanced future phase.

11. Appendix

11.1 Revision History

Version	Date	Author	Description
1.0	August 2026	Student Development Team	Initial SRS document for SkyCast